

Experiment No. – 6

Aim –

- (A.) A website maintains records of users who have registered on the platform and the status of confirmation requests sent to them. A confirmation request can either be successfully completed or may expire without completion.

For each registered user, calculate the confirmation rate, which is defined as:

$$\text{Confirmation Rate} = \frac{\text{Number of Successful Confirmations}}{\text{Total Confirmation Requests}}$$

- (B.) A ride-booking company maintains information about trip requests and the users involved in those trips. Some users may be restricted from using the platform, while others are active participants. Each trip can either be completed successfully or cancelled by one of the participants.

For each day between **2013-10-01** and **2013-10-03**, determine the proportion of canceled trips among all valid trips. A trip should be considered valid only when both the customer and the driver associated with that trip are active users. The final result should display the date and the calculated cancellation rate rounded to two decimal places

Tools Required

- Postgres
- PgAdmin4

Queries:

A.

```
CREATE TABLE Sales(  
    sale_date DATE,  
    fruit VARCHAR(20),  
    sold_num INT,  
    PRIMARY KEY (sale_date, fruit)  
);  
  
INSERT INTO Sales  
VALUES  
('2020-05-01','apples',10),  
('2020-05-01','oranges',8),
```

```

('2020-05-02','apples',15),
('2020-05-02','oranges',15),

('2020-05-03','apples',20),
('2020-05-03','oranges',0),

('2020-05-04','apples',15),
('2020-05-04','oranges',16);

select sale_date , sum(
    CASE
        WHEN fruit = 'apples' then sold_num
        Else -sold_num
    end
) as diff
from Sales
group by sale_date
Order by sale_date;REFERENCES Signups(user_id)
);

```

Outputs:

	sale_date date 	diff bigint 
1	2020-05-01	2
2	2020-05-02	0
3	2020-05-03	20
4	2020-05-04	-1

Learning Outcome

After performing this SQL query, the following learning outcomes were achieved:

1. **Learned to create and populate a database table** using CREATE TABLE and INSERT INTO statements.
2. **Understood the use of a composite primary key** (sale_date, fruit) to uniquely identify each sales record.
3. **Learned to use the CASE statement** for applying conditional logic within an SQL query.
4. **Used the SUM() aggregate function** to calculate the difference between the number of apples and oranges sold on each date.
5. **Understood the use of GROUP BY** to perform calculations separately for every sale_date.
6. **Learned to use ORDER BY** to display the results in chronological order.